ACADEMIC TECHNICAL PRESENTATION • AUTOMOTIVE SOFTWARE & AI SYSTEMS

AUTOSAR AI

High-Level Design Document Intelligence Platform

"Local-First AI for Evidence-Grounded Automotive Architecture Analysis"

STUDENT & PROJECT SPECIFICATIONS

Candidate Name: **Hamdhan Ahmed B**

Registration Number: **127003081**

Domain: **Automotive Software Engineering & AI Architectures**

System Paradigm: **Local-First LLM (Ollama llama3.2:3b) + Persistent Vector Store**

Verification Standard: **Strict Source Grounding & Verifiable Citations**

PROJECT REPOSITORY & CODEBASE (CLICK TO OPEN)

https://github.com/hamdhan05/AUTOSAR-AI-Implementation

Full Stack Implementation • Automated Unit & Integration Tests • Docker Ready

OFFLINE-FIRST AIR-GAPPED PRIVACY

Ollama (llama3.2:3b) local inference; zero automotive IP leaves local premises.

VERIFIED TEST SUITE (100% PASS RATE)

28 / 28 automated tests passing across 12 test modules on Python 3.11.

ENGINEERING CHALLENGE & STRATEGIC GOALS

Problem Statement and Project Objectives

Overcoming verification bottlenecks in massive automotive AUTOSAR High-Level Design specifications

THE ENGINEERING PROBLEM IN AUTOMOTIVE HLD

Massive Specification Volume & Slow Navigation

AUTOSAR High-Level Design (HLD) specifications span hundreds of dense pages containing deeply nested architectural models. Manual information retrieval requires substantial engineering hours and delays review cycles.

Fragmented Architectural Traceability

Identifying Software Components (SWCs), Runnables, Ports, and Interface mappings across disparate sections is error-prone. Critical dependencies and signal types are easily overlooked during manual inspection.

Revision Comparison Drift & Hallucination Risk

Comparing specification revisions line-by-line is tedious and risks silent architecture drift. Meanwhile, generic public cloud LLMs hallucinate non-existent interfaces and lack document grounding.

CORE PROJECT OBJECTIVES & SOLUTIONS

1. Automated Semantic HLD Ingestion

Ingest multi-page PDF specifications, extract text structure page-by-page, and build persistent searchable vector indexes with strict document & page metadata.

2. Evidence-Grounded Natural-Language Q&A

Allow automotive architects to query complex component behaviors in natural English, retrieving top-k relevant excerpts via local RAG with zero data leakage.

3. Strict Traceability & Abstention Guardrails

Provide transparent document, section, and page citations for every assertion, with strict LLM abstention directives when source evidence is insufficient.

4. Architectural Validation & Revision Diffing

Automatically extract SWCs, Runnables, Ports, and Signals; run enterprise rule checks (circular dependencies, insecure IPC); compute baseline-vs-revision deltas.

CORE ENGINEERING PRINCIPLE: AUTOSAR AI acts as an authoritative analysis assistant and verification co-pilot—accelerating specification discovery while strictly preserving human engineer oversight, review sign-off, and final architectural approval.

THREE-TIER MULTI-SERVICE TOPOLOGY

System Architecture & Communications Flow

Decoupled architecture separating presentation, orchestration, persistence, and local AI inference

PRESENTATION LAYER

Streamlit Multipage Web UI • localhost:8501

- **Dashboard (Page 1):** Document & entity metrics, subsystem diagnostics
- **Documents (Page 2):** PDF upload, SHA-256 deduplication, lifecycle tracking
- **Architecture Explorer (Page 3):** SWC, Runnable, Port, Signal tables & reviews
- **AI Assistant (Page 4):** RAG conversation, citation inspector, verification
- **Comparisons (Page 5):** Baseline vs revision diffs, rename detection
- **Findings (Page 6):** Circular dependencies, insecure IPC resolution
- **Reports & Settings (Page 7):** PDF/CSV/JSON exports, diagnostic health checks

HTTP REST

BACKEND ORCHESTRATION

FastAPI Asynchronous REST API • localhost:8000

- **Document Ingestion Service:** PyMuPDF parsing, bookmark extraction, chunking
- **Hybrid Retrieval Engine:** ChromaDB dense vectors + BM25 keyword + RRF
- **LLM Orchestrator Service:** Context assembly, prompt formatting, abstention
- **Entity & Rule Validator:** Extracts SWC/Ports/Signals; circular dependency checks
- **Grounding & Claim Service:** Claim segmentation & independent evidence checks
- **Comparison & Diff Engine:** Computes structural deltas & rename detection
- **Report Generation Service:** ReportLab PDF generation & CSV sanitization

SQL / IPC REST

DATA & LOCAL AI LAYER

Zero Cloud Dependency • 100% Offline

- **SQLite (autosar_ai.db)**

  SQLAlchemy ORM + Alembic migrations

- **ChromaDB (Persistent)**

  Persistent HNSW embeddings in ./data/vector_store

- **all-MiniLM-L6-v2**

  SentenceTransformers (384-dimensional dense vectors)

- **Ollama (llama3.2:3b)**

  Local server at localhost:11434; zero telemetry

- **File System Storage**

  Raw PDFs & generated exports in ./data/uploads & exports

ARCHITECTURE HIGHLIGHTS: **Strict Multi-Process Isolation:** Streamlit (8501) ↔ FastAPI (8000) ↔ Ollama (11434) run as independent processes. **Air-Gapped Operation:** Zero external API dependencies; robust privacy for proprietary automotive IP. **Reproducible:** Docker & docker-compose enabled.

DATA PROCESSING PIPELINE

Document Ingestion and Indexing Workflow

Structured multi-stage transformation from raw automotive PDF specifications to searchable knowledge

STEP 1
Upload & Validate
PDF Upload

- Engineer uploads HLD PDF via Streamlit UI
- File extension & 50MB size limit validation
- SHA-256 fingerprint computed for duplicate check
- Document record set to PENDING in SQLite

STEP 2
Parse & Extract
PyMuPDF Engine

- PyMuPDF (fitz) extracts text page-by-page
- Physical page numbers preserved for citations
- TOC & bookmark hierarchy extracted
- Heading context associated with passages

STEP 3
Chunk & Window
Sliding Window

- Sliding window text partitioning
- Chunk size: 1,000 characters
- Chunk overlap: 150 characters
- Preserves sentence and section semantics
- Generates unique chunk hashes

STEP 4
Embeddings
Dense Vectors

- SentenceTransformers all-MiniLM-L6-v2
- Generates 384-dimensional dense vectors
- Attaches document_id, version_label, page_start, page_end
- Runs locally on CPU/GPU

STEP 5
Dual Storage
Persistent Stores

- Vectors & metadata saved in ChromaDB
- Structured chunk records in SQLite database
- Relational foreign keys linked to Document
- Document status updated to READY

STEP 6
Extraction
Entity Discovery

- Regex heuristics extract:
  - Software Components (SWC)
  - Runnables (Runnable_*)
  - Interfaces (*_IF)
  - Ports (PPort / RPort)
  - Signals (Signal_*, types)

IMPLEMENTATION STATUS & ARCHITECTURAL SCOPE NOTE: Current implementation actively extracts Software Components, Runnables, Interfaces, Ports (PPort/RPort), and Signals via pattern-based heuristic extractors with confidence scores (0.88–0.92). Granular AST-based ARXML schema validation, complex bus packaging, and schedule table extraction are designated for Phase 2.

RETRIEVAL-AUGMENTED GENERATION ARCHITECTURE

AI Assistant and RAG Working

Local inference pipeline delivering evidence-grounded answers with verifiable citations and abstention safeguards

STAGE 1: RETRIEVAL

Hybrid Semantic Search

- Engineer enters natural-language engineering query in UI
- Query encoded into 384-d vector via all-MiniLM-L6-v2
- ChromaDB performs dense cosine vector search (top_k=5)
- BM25 keyword search scores candidate document passages
- Reciprocal Rank Fusion (RRF) merges dense and keyword ranks
- Extracts top hits with document, page, and section metadata

STAGE 2: PROMPTING

Context Assembly

- FastAPI builds strict automotive architecture system prompt
- Assembles top-k retrieved chunks into structured context window
- Enforces explicit citation tagging: [Doc: HLD_v1.0, Page 4]
- Strict Abstention Mandate: If evidence is insufficient, LLM must state lack of evidence rather than hallucinating

STAGE 3: INFERENCE

Local LLM Synthesis

- Dispatches assembled context prompt to local Ollama runtime
- Active model: llama3.2:3b (or Qwen2.5:3b) at localhost:11434
- Low-temperature deterministic inference (~0.2 temperature)
- Zero telemetry or cloud traffic; completely air-gapped
- Synthesizes grounded technical answer referencing citations

STAGE 4: VALIDATION

Grounding & Display

- Claim Segmentation breaks response into individual assertions
- Independent claim verification matches assertions to excerpts
- Computes Grounding Score: (Grounded Claims / Total Claims) %
- UI renders response with clickable citation source excerpts
- Flags 'Review Required' badge if Grounding Score is below 80%

Evidence-Grounded Synthesis

Strict System Prompt constraint: LLM answers only using retrieved passages. If asked about components outside the document, it refrains from speculating.

Granular Traceability Citations

Every assertion is supported by document name, version label, and exact physical page number, allowing engineers to verify claims against the original PDF in seconds.

Deterministic Abstention Safeguard

When retrieved context has similarity below threshold (< 0.5) or lacks key entities, the assistant outputs: 'The available documents do not provide sufficient evidence.'

IMPLEMENTED APPLICATION MODULES

Comprehensive Functional Modules

Six verified functional subsystems spanning document lifecycle, analysis, validation, and enterprise reporting

01. DASHBOARD & HEALTH DIAGNOSTICS
Centralized Metrics & Subsystem Telemetry

- Aggregated document counts, total chunk volume, and entity breakdown
- Active LLM provider diagnostics: model name, base URL, latency probe
- Vector store health indicator and ChromaDB collection status
- Audit logs tracking recent ingestion, queries, and review actions

02. DOCUMENT REPOSITORY & LIFECYCLE
PDF Ingestion, Deduplication & Management

- Single/batch PDF upload with real-time file size & type validation
- SHA-256 fingerprint deduplication prevents duplicate indexing
- Background processing lifecycle: PENDING → PROCESSING → READY
- Document metadata inspector (page count, upload timestamp, version)

03. ARCHITECTURE EXPLORER
Extracted Components, Ports & Review Actions

- Categorized catalog: Software Components, Runnables, Interfaces, Ports, Signals
- Confidence scoring per extracted entity (0.88 to 0.92)
- Source excerpt & physical page number mapping for auditability
- Interactive engineer review actions (Approve, Reject, Flag for Review)

04. AI ASSISTANT (RAG ENGINE)
Natural-Language Q&A with Evidence Citations

- Conversational interface grounded strictly in indexed HLD documents
- Document-scoped querying allowing filtering by specific HLD version
- Clickable citation pills displaying exact source excerpts and pages
- Claim verification drawer displaying Grounding Score and confidence

05. COMPARISONS & REVISION DIFF
Baseline vs Revision Architectural Deltas

- Automated structural comparison between document versions (e.g., v1.0 vs v2.0)
- Categorizes architectural deltas: Added, Removed, and Modified entities
- Signal data type change detection (e.g., uint16 modified to uint32)
- Entity rename detection tracking modified component designations

06. FINDINGS, VALIDATION & REPORTS
Automated Rule Checks & Multi-Format Exports

- Rule engine detects circular dependencies, insecure IPC, unbounded arrays
- Unresolved interface reference identification and signal type conflicts
- One-click ReportLab PDF generation, structured JSON, and sanitized CSV exports
- CSV formula injection defense (=, +, -, @ sanitized) preventing spreadsheet exploits

Verified Scope: All 6 modules are operational in the existing codebase with corresponding Streamlit pages and FastAPI endpoints. Advanced ARXML round-tripping and multi-agent reasoning are classified as future scope.

EMPIRICAL VALIDATION & TEST RESULTS

Testing, Verification & Quality Assurance

Empirically verified test results across all functional modules, security controls, and RAG retrieval pipelines

28 / 28
Automated Tests Passed
100% Pass Rate across all 12 test suites

0
Test Failures / Errors
Zero regressions detected in verification audit

35
Evaluation Queries
Curated benchmark dataset for retrieval evaluation

100%
Offline Capable
Verified local execution with zero internet connectivity

Document Ingestion & Chunking Pipeline (5 Tests Passed)

- test_chunking_and_entity_extraction: Verifies 1000-char chunking with 150-char overlap
- test_full_document_ingestion_and_cleanup: Ingests PDF, tests ChromaDB indexing & deletion
- test_create_project, test_get_projects, test_upload_invalid_file: API input validation

Entity, Port/Signal & Rule Validation (6 Tests Passed)

- test_ports_and_signals_extraction: Confirms PPort/RPort, direction & typed signals (float32)
- test_mermaid_generation_and_dependency_traversal: Verifies graph dependency generation
- test_findings_and_entity_review: Validates circular dependency and insecure IPC rule triggers

Local Ollama LLM, Grounding & Traceability (7 Tests Passed)

- test_ollama_connection_diagnostic & test_active_provider_info: Confirms Ollama health probe
- test_ask_question_insufficient_evidence: Verifies deterministic abstention guardrail
- test_claim_segmentation, test_evidence_verification, test_grounding_score: Verifies claim checks

Revision Comparison, Reports & Security (10 Tests Passed)

- test_csv_formula_injection_sanitization: Sanitizes '=cmd', '+', '-', '@' against injection
- test_rename_detection_and_data_type_diff: Detects entity renames and signal type deltas
- test_report_generation_and_download, test_governance, test_audit: Validates export workflows

VERIFIED END-TO-END WORKFLOW EXAMPLE: AUTOSAR HLD PDF Upload (Page 4) → PyMuPDF Extraction → ChromaDB Embedding Indexing → Engineer Query: 'What interfaces does MotorControl_SWC provide?' → Top-k Retrieval → Local Ollama Inference → Verified Cited Answer with Page 4 excerpt and 92% Grounding Score.

ROADMAP & STRATEGIC CONCLUSION

Four-Phase Evolution & Strategic Conclusion

From functional prototype to an enterprise-grade automotive architecture intelligence platform

PHASE 1
Traceability & Governance

CURRENT BASELINE & REFINEMENTS

- Formal document version tagging & governance metadata
- Claim-level answer segmentation & independent verification
- Inline Markdown, JSON, and sanitized CSV exports
- Document-scoped query filtering
- Grounding Score & Review Recommendation badge

PHASE 2
Architecture Depth

TOPOLOGY & SCHEMA EXPANSION

- Full AST-based Port, Signal & Interface extraction
- Interactive architecture topology graphs via Mermaid & Cytoscape
- Dependency traversal & change-impact propagation
- Bidirectional ARXML schema round-tripping & synchronization
- Bus frame packaging & signal timing analysis

PHASE 3
Enterprise Governance

COMPLIANCE & TEAM COLLABORATION

- Formal engineer sign-off workflow for AI-assisted analysis
- Immutable audit trails & persistent Q&A review history
- Per-component architectural compliance scorecards
- Multi-user role-based access control (RBAC)
- Integration with JIRA and enterprise PLM tools

PHASE 4
Evaluation & Multi-Agent

BENCHMARKING & ADVANCED AGENTS

- Automated benchmark execution against 35-query dataset
- Retrieval metrics tracking (MRR, Hit Rate@1, latency)
- Hybrid BM25 + Dense retrieval dynamic weight tuning
- Multi-agent architectural critique & consistency validation
- ISO 26262 functional safety traceability mapping

STRATEGIC CONCLUSION: AUTOSAR AI successfully bridges the gap between massive, static High-Level Design PDF specifications and dynamic engineering workflows. By uniting local RAG retrieval, rule-based architecture validation, automated revision diffing, and strict citation traceability, the platform provides automotive architects with verifiable, evidence-grounded insights while preserving complete offline data sovereignty.

Hamdhan Ahmed B | Registration No: 127003081 | GitHub: https://github.com/hamdhan05/AUTOSAR-AI-Implementation